Data-Driven Recommendations for H&M: A Study on Candidate Generation and Learning-to-Rank

Group 3: Thành Uyển Dung, Vũ Ngọc Dương, Đỗ Ánh Ly, Nghiêm Trà My, Tô Bích Ngọc, Lê Thị Anh Thư
Course: DDM - Data-Driven Marketing
Email: 11230522@st.neu.edu.vn, 11230526@st.neu.edu.vn, 11230563@st.neu.edu.vn,
11230572@st.neu.edu.vn, 11230577@st.neu.edu.vn, 11230591@st.neu.edu.vn

***Abstract*—This paper addresses the problem of large-scale fashion recommendation using a data-driven approach on the H&M Personalized Fashion Recommendations dataset (31.8M transactions, 1.37M customers, 105.5K articles). We develop and compare a family of recommender models, including popularity baselines, item-based collaborative filtering, re-purchase heuristics, and a two-stage candidate-generation plus LightGBM LambdaRank pipeline. The proposed two-stage model achieves a MAP@12 of 0.0334, a 38.6% improvement over the strongest single-strategy baseline (repurchase) and a 5x improvement over the global-popularity baseline. The results demonstrate statistically and practically significant improv-ements in the key business metrics MAP@12. Based on both statistical and business significance, we provide actionable recommendations for system-wide deployment.**

***Index Terms*—Recommender Systems, Gradient Boosting, Learning-to-Rank, Candidate Generation, Fashion**

I. Introduction

The application of recommender systems in fashion retailing has proven to be an effective way to improve customer satisfaction, increase sales, and create a more engaging shopping experience [1]. Fashion is a highly subjective and dynamic domain [2], where personal taste and fashion trends change rapidly [3], making accurate personalization both valuable and technically challenging. This study started from a Kaggle Competition proposed by the H&M brand and aims to explore the transactions, customers and article data of H&M and build an effective recommender system.

The core business question is: *given a customer's purchase history and profile, which 12 articles is the customer most likely to buy in the next seven days?* The dataset exhibits the canonical pathologies of e-commerce data at industrial scale: extreme item-popularity skew, severe user-activity skew, large cold-start populations, a rapidly turning catalogue, and strong seasonality. We address these challenges with a two-stage architecture that decouples *candidate generation* (recall-oriented) from *ranking* (precision-oriented), which is the industry standard for recommendation at scale. The final ranker is a LightGBM LambdaRank model trained on 33 engineered user, article, and user–article interaction features. Performance is measured by MAP@12 (Mean Average Precision at 12).

The remainder of this paper is organized as follows. Section II reviews relevant literature on recommender systems and learning-to-rank methods. Section III presents exploratory data analysis. Section IV describes the modeling approach, including baselines, collaborative filtering, and the two-stage LightGBM ranker. Section V details the experimental design and evaluation protocol. Section VI reports results and discusses error patterns. Section VII concludes and outlines deployment recommendations.

II. Literature Review

Recommender systems have become a central component of modern e-commerce platforms, enabling firms to personalize user experiences, increase engagement, and improve conversion rates [4]. The literature on recommender systems is extensive and can broadly be categorized into three main streams: (i) traditional recommendation approaches, (ii) domain-specific challenges in fashion recommendation, and (iii) large-scale industrial recommender architectures based on multi-stage retrieval and ranking

A. Traditional Recommender System Approaches

Early recommender systems primarily relied on collaborative filtering (CF) and content-based filtering techniques. Collaborative filtering methods, including user-based and item-based approaches, leverage historical interaction data to identify similarities between users or items and generate recommendations accordingly [5]. Among these, item-based collaborative filtering has been widely adopted in industry due to its scalability and interpretability. For example, similarity measures such as cosine similarity or Jaccard similarity are commonly used to infer relationships between items based on co-purchase patterns.

Content-based methods, on the other hand, recommend items by matching user preferences with item attributes. These approaches rely on structured metadata such as product categories, colors, or textual descriptions. While content-based systems are effective in handling new items (cold-start on the item side), they often suffer from limited diversity and over-specialization [6].

Hybrid models combine collaborative filtering and content-based techniques to overcome the limitations of each approach. As highlighted by Adomavicius and Tuzhilin [1], hybrid systems can improve recommendation accuracy and robustness by integrating multiple sources of information. However, traditional methods generally struggle when applied to large-scale datasets with millions of users and items, where computational efficiency and data sparsity become critical challenges.

B. Challenges in Fashion Recommendation

The fashion domain presents unique challenges that distinguish it from other recommendation settings. First, user preferences in fashion are highly subjective and dynamic, influenced by personal taste, cultural factors, and rapidly changing trends. Unlike domains such as movies or music, where preferences tend to be relatively stable, fashion consumption exhibits strong temporal dynamics and seasonality [7].

Second, the fashion industry is characterized by rapid catalogue turnover. New items are introduced frequently, while existing products are discontinued after short lifecycles. This creates a severe item cold-start problem, as many products have little or no historical interaction data. As noted in prior work on fashion recommendation, the short lifespan of items significantly reduces the effectiveness of traditional collaborative filtering approaches [8].

Third, repeat purchase behavior plays a critical role in fashion retail. Customers often repurchase similar items (e.g., the same product in different colors or sizes), leading to strong intra-product correlations. This behavioral pattern suggests that simple heuristics such as repurchase-based recommendations can serve as strong baselines, especially in datasets with high levels of user loyalty [9].

Finally, data sparsity and user heterogeneity further complicate the recommendation task. A large proportion of users exhibit very limited interaction histories, making it difficult to learn personalized preferences. This imbalance between highly active users and sparse users is a well-documented phenomenon in e-commerce datasets and poses a significant challenge for model design [10].

C. Two-Stage Recommendation Architectures

To address scalability and performance issues in large-scale systems, modern recommender systems increasingly adopt a two-stage architecture consisting of candidate generation (retrieval) followed by ranking. This paradigm is widely used in industrial applications such as Amazon, YouTube, and other large e-commerce platforms [11], [12].

In the first stage, a retrieval model generates a relatively small set of candidate items (typically a few hundred) from a very large item pool. This stage prioritizes recall, ensuring that relevant items are included in the candidate set. Common techniques include heuristic-based methods (e.g., popularity, repurchase), collaborative filtering, and embedding-based retrieval models.

In the second stage, a ranking model reorders the candidate items based on predicted relevance scores. Learning-to-rank algorithms, such as LambdaRank and gradient boosting decision trees, are widely used due to their strong performance and ability to incorporate heterogeneous features [13]. These models optimize ranking-specific objectives (e.g., NDCG or MAP), making them particularly suitable for recommendation tasks where the order of items is critical.

The separation of retrieval and ranking offers several advantages. First, it significantly reduces computational complexity by limiting expensive scoring operations to a small candidate set. Second, it allows different models to be optimized independently for recall and precision. Third, it enables the integration of domain knowledge through heuristic candidate sources, which can complement data-driven ranking models.

D. Learning-to-Rank Methods in Recommendation

Learning-to-rank (LTR) techniques have been extensively applied in recommendation systems to optimize the ordering of items presented to users. Among these, LambdaRank, implemented in gradient boosting frameworks such as LightGBM, has gained popularity due to its efficiency and effectiveness in handling large-scale tabular data [14].

LTR models operate on user–item pairs and leverage a rich set of features, including user attributes, item characteristics, and interaction signals. Prior studies have shown that interaction-based features, such as recency and frequency of past purchases, are particularly strong predictors of future behavior [15]. Additionally, short-term popularity and trend-based features are often more informative than long-term aggregates in dynamic domains like fashion.

Compared to deep learning approaches, gradient boosting models offer several practical advantages, including faster training, easier interpretability, and strong performance on structured data. As a result, they remain a competitive choice in industrial recommendation pipelines, particularly when feature engineering plays a central role.

E. Research Gap

While prior work has explored various recommendation techniques, there remains a need for models that effectively combine domain-specific behavioral insights with scalable machine learning architectures. In particular, the integration of repurchase behavior, short-term popularity, and demographic segmentation within a unified framework has not been fully explored in the context of large-scale fashion datasets.

This study addresses this gap by proposing a two-stage recommender system that combines multiple heuristic candidate sources with a LightGBM LambdaRank model. The design is explicitly informed by exploratory data analysis, allowing the model to capture key characteristics of fashion consumption, including repeat purchasing, non-stationary trends, and user heterogeneity. By grounding the modeling approach in both data-driven insights and established recommender system principles, this work contributes to the development of effective and practical solutions for large-scale fashion recommendation.

III. Exploratory Data Analysis

The dataset is comprised of three tables covering the period **2018-09-20 to 2020-09-22**: a transactions log (31.8M rows; columns t_dat, customer_id, article_id, price, sales_channel_id), a customer table (1.37M rows; demographic and membership attributes), and an article catalogue (105.5K rows; 25 metadata fields covering product hierarchy, colour, section, and department). All identifiers were cast to

compact integer dtypes to reduce memory consumption by approximately 70%.

A. Customer Behaviour and Activity Skew

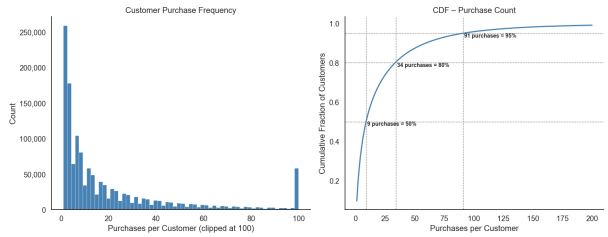


Figure 1. Customer Purchase Frequency

Customer activity is highly skewed, posing a significant challenge for collaborative filtering due to data sparsity. As shown in Figure 1, the median customer has only 9 transactions over the two-year window, and 80% of customers have fewer than 34 total purchases. This long tail of low-frequency users suggests a recommendation strategy that balances historical preferences with global popularity trends.

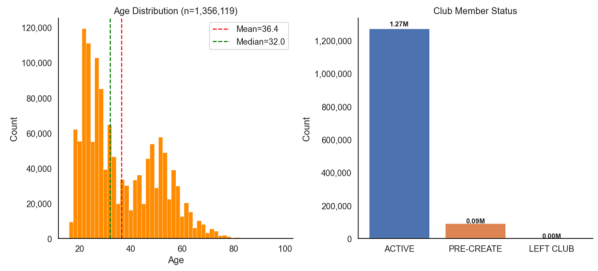


Figure 2. Customer Profile

As illustrated in Figure 2, the age distribution is approximately log-normal with a median of 32 years, centering on a young-adult core. Additionally, the broad age range (16 to 84) suggests that age-based segmentation—further analyzed in subsection F—could provide a critical insights for the recommendation engine. Beyond age, customer engagement is captured through membership indicators: about 93% of customers are active club members, and a considerable fraction of the FN/Active membership flags are missing. However, we treat them as a meaningful categorical level rather than as missingnes.

B. Repurchase Behavior Across Product Attributes

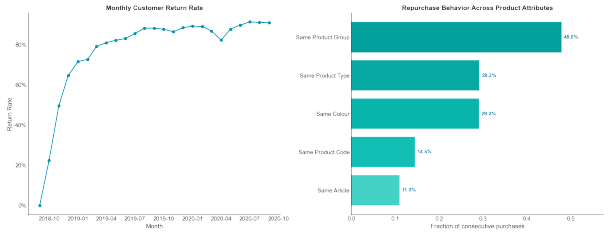


Figure 3. Customer Loyalty: Repurchase Analysis

The monthly retention curve in Figure 3 reveals that 60–80% of active customers each month are returning users, indicating substantial habitual purchasing behavior. To understand the nature of these repeat transactions, we examine attribute-level consistency across consecutive purchases. Interestingly, users tend to repurchase the same articles or, more frequently, the same product code in a different colour or size. Indeed, nearly 50% of consecutive purchases remain within the same product group, while approximately 30% preserve product type or color attributes. This strong preference consistency has motivated a dedicated repurchase candidate source in Section III and explained why a naive "recommend what the user already bought" heuristic dominates all popularity baselines.

C. Catalogue Dynamics and Discontinuation

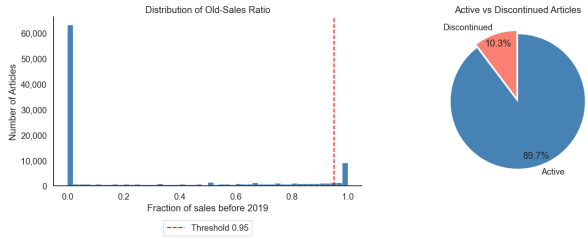


Figure 4. Distribution of active vs. discontinued articles in the H&M catalogue.

Figure 4 shows that the catalogue turns over rapidly. Using a simple rule – articles for which larger than or equal 95% of lifetime sales occurred before 2019 – we identify 10,778 discontinued articles ($\approx 10.3\%$ of the catalogue) that are still occasionally retrieved by naive popularity strategies but should be filtered out at serving time. We refer to this as the out-of-stock mask and apply it uniformly in candidate generation.

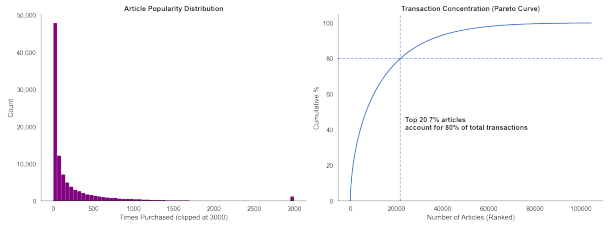


Figure 5. Article Popularity Analysis

Furthermore, Figure 5 reveals another fact about article: most of transactions are concentrated among a few top items - the top 20.7% of popular articles account for 80% of total transactions. Thus, a recent pop-ularity is far more informative than all-time popularity. Indeed, A baseline that recommends the global top-12 over the last two weeks achieves MAP@12 = 0.0068, more than 2.3x the all-time top-12 baseline (0.0029). This confirms that fashion preferences are non-stationary and motivates using short-term data in the ranker.

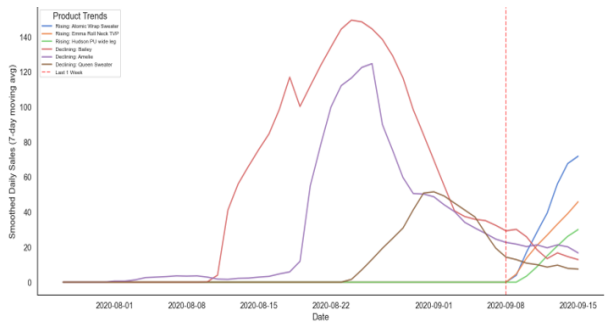


Figure 6. Article Sales Trend Analysis

However, not all popular articles are the same; we further differentiate them using a trend score in Figure 6, which compares recent sales to their historical baseline. This allows us to distinguish between consistently popular items and newly rising products that reflect current demand shifts. As we can see in the figure, rising items exhibit a clear upward trajectory in recent weeks, with sharp increases in smoothed daily sales, while declining items peak earlier and steadily lose momentum over time. Thus, features related to the trend score (e.g., art_trend_score), together with popularity signals (e.g., art_pop_1w), emerge as some of the most important predictors in the recommendation model, as confirmed by the feature importance analysis in Figure 12.

D. Recency-Driven Purchase Behavior

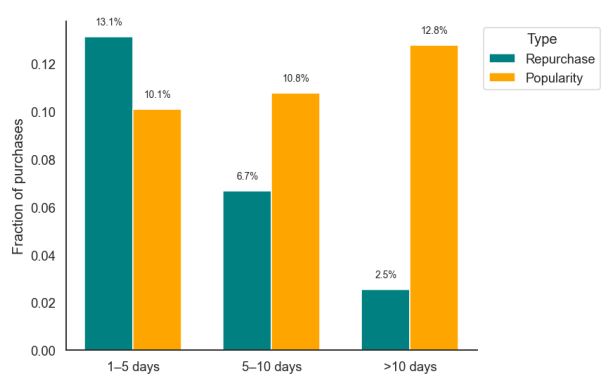


Figure 7. Repurchase vs. Popularity Preference Across Recency Levels

As discussed earlier in part B and C, repurchase items and popular articles are one of the main sources for our recommendation system. However, to determine which source should be prioritized, we compare the share of purchases between repurchase and popular items across different recency groups. An interesting pattern emerges: users with low recency (1–5 days) show a strong preference for repurchase items, while those with higher recency (>10 days) increasingly rely on popular products. This suggests a shift from personalized, history-driven behavior to more general, popularity-driven exploration as user inactivity increases. Therefore, recency-related features (e.g., ua_days_since_last_tx) are included and play a crucial role in adapting recommendation strategies. Indeed, these feature are among the most important features in the model, as shown in 12.

E. Price Sensitivity and Sales Channel Analysis

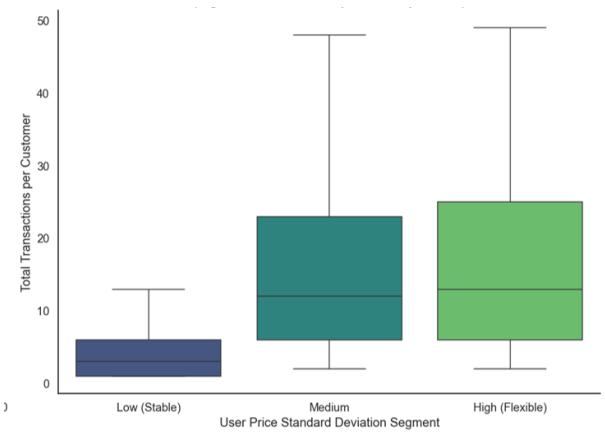


Figure 8. Price Flexibility vs. Purchase Volume

Another purchasing pattern emerges when examining users' price sensitivity, measured by price standard deviation. We segment customers into three equally-sized groups based on price flexibility and compare their total transactions. Figure 8 reveals that price-flexible users (high STD) exhibit substantially higher purchase volumes and greater variance compared to price-stable users, suggesting that exploration-oriented customers engage more actively with diverse product ranges. This single observation implies that low-sensitivity users could receive diverse recommendations mixing repurchase, trending, and popular items across price ranges, whereas high-sensitivity users require more price-consistent suggestions aligned with their spending patterns.

Figure 9. Sales Channel Distribution and Price Patterns

To further analysis, price sensitivity is combined with sales channel analysis in Figure 9. It is shown that H&M

operates through online (70.4%) and offline stores (29.6%). Notably, online transactions demonstrate higher mean prices and slightly lower price sensitivity. This suggests that online users typically exhibit higher purchase frequency and average spending, indicating that recommendation systems can prioritize premium and diverse product suggestions for online channels. Thus both price and sale-channel related features are beneficial for recommendation strategies, as Figure 12 shows that features like user_online_ratio and user_price_std rank among the top feature importances in the model.

F. Demographics, Seasonality and Segmentation

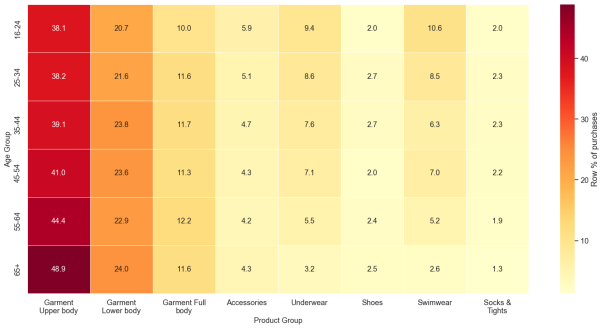


Figure 10. Percentage of Purchase by Age Group x Product Group (Top 8 groups)

Age–product cross-tabulations reveal a clear shift in purchasing behaviour across the lifecycle. Customers aged 16–24 allocate approximately 38% of purchases to upper-body garments and exhibit relatively diverse spending across categories such as swimwear and underwear. In contrast, customers aged 45+ show increasingly concentrated behaviour, with the share of upper-body garments rising to nearly 49% for the 65+ group, alongside a marked decline in discretionary categories. Notably, categories such as swimwear and underwear steadily decrease with age, while core apparel (upper and lower body) becomes more dominant, indicating a transition toward more functional and less exploratory purchasing patterns. Thus, we bin age into six groups (16–24, 25–34, 35–44, 45–54, 55–64, 65+) and use the bin as both a candidate-generation key and a ranker feature.

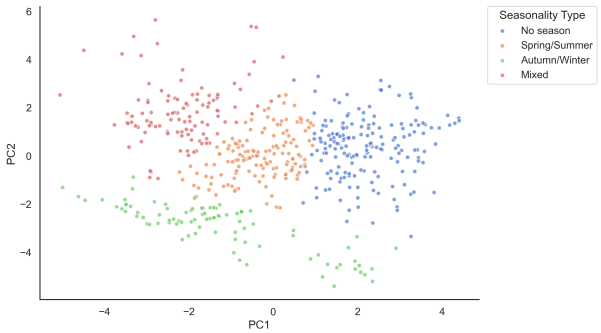


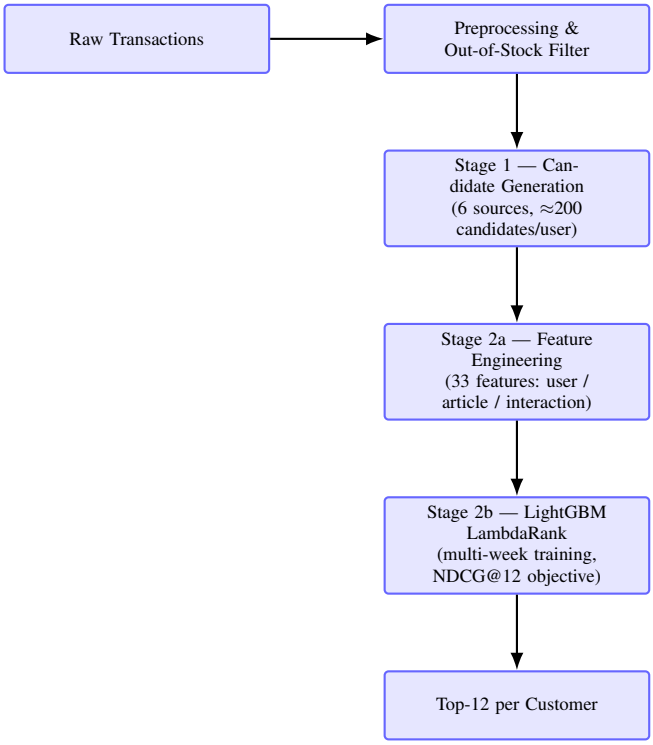
Figure 11. PCA of Category Sale Trends-GMM Clusters (Seasonality)

Price seasonality was investigated by clustering each product group's weekly average-price series via PCA followed by Gaussian Mixture Modelling, which surfaced four distinct seasonal patterns, consistent with H&M's spring/summer/ autumn/winter merchandising cycles. We do not model seasonality directly but exploit it implicitly through short-window popularity and trend features.

IV. Modeling

A. Architecture Overview

We adopt a two-stage retrieve-then-rank architecture. Stage 1 (candidate generation) retrieves a small set of ≈ 200 plausible articles per user from six complementary heuristic sources, achieving high recall at low computational cost. Stage 2 (re-ranking) scores every resulting (user, candidate) pair with a LightGBM LambdaRank model trained on 33 hand-crafted features. The final recommendation is the top-12 candidates by predicted score.



This design is better than using a monolithic collaborative-filtering or neural model for three complementary reasons. First, exhaustively scoring all 105.5K articles for each of the 1.37M customers in a single pass is computationally infeasible at serving time. Second, the candidate sources encode strong, interpretable patterns (e.g., "users often buy the same basic items again") that a purely learned model would have to rediscover from scratch. Third, the two stages can be tuned independently: Stage 1 is tuned for recall (no relevant item should be excluded before Stage 2 sees it), and Stage 2 is tuned for precision (the top-12 should contain as many actually-purchased articles as possible). Jointly optimizing both objectives in a single model requires expensive approximate nearest-

neighbor infrastructure; decoupling them keeps the pipeline transparent and maintainable.

B. Preprocessing and Out-of-Stock Filtering

Before any modeling begins, three preprocessing steps are applied uniformly. First, all 64-bit float and integer columns are downcast to 32-bit equivalents, reducing RAM consumption by approximately 40% — essential for working with a 31.8M-row transactions table in a single process. Second, customer profiles with missing `age` fields ($\approx$ 20% of rows) are imputed to the dataset median; missing `club_member_status` and `fashion_news_frequency` values are set to explicit sentinel categories (`UNKNOWN` and `NONE`) rather than being dropped, preserving their structural information for the ranker. Third, the out-of-stock mask is constructed: any article for which $\geq$ 95% of its lifetime sales occurred before 2019-01-01 is classified as discontinued. This criterion identifies 10,781 articles ($\approx$ 10.2% of the active catalog) that are historically popular but should not be recommended. The mask is applied to every candidate source before union, ensuring discontinued items can never reach the ranker.

C. Stage 1 - Candidate Generation

For each user, up to 200 candidate articles are assembled from six sources, deduplicated by `article_id`, and passed to Stage 2. Sources are evaluated in priority order so that when the total exceeds 200, the highest-signal sources are retained first.

1) **Repurchase-short**
   - Lookback window: Last 14 days
   - Cap per user: Unlimited
   - Primary signal: Strong recency — items the user is actively buying
2) **Repurchase-long**
   - Lookback window: Last 60 days
   - Cap per user: 50
   - Primary signal: Habitual/staple purchases
3) **Product-code repurchase**
   - Lookback window: Last 60 days
   - Cap per user: 30
   - Primary signal: Same `product_code` with the ones the user has bought (same garment, different color or size)
4) **Global recent popularity**
   - Lookback window: Last 14 days
   - Cap per user: 100
   - Primary signal: Trending items across all customers
5) **Age-segment popularity**
   - Lookback window: Last 14 days
   - Cap per user: 30
   - Primary signal: Trending within the user's age cohort
6) **Category popularity**
   - Lookback window: Last 14 days
   - Cap per user: 20 per category
   - Primary signal: Trending within the user's top-3 most-purchased product groups

The first three sources exploit the repurchase signal highlighted in Section III-B: if a user has bought an article or a variant of it recently, it is the single strongest predictor of their next purchase. The latter three sources serve as recall safety nets for customers who have a short history or who are being served a trending item outside of their prior catalog. Age bins are the same six groups used throughout the study (16–24, 25–34, 35–44, 45–54, 55–64, 65+), and category popularity is computed over each user's three most-purchased product_group_name values.

Raising the candidate budget from 50 to 200 (moving from model v1 to v2) increased held-out recall@200 from 9.6% to 28%, directly raising the upper bound on achievable MAP@12. A candidate that never enters Stage 1 can never be recommended regardless of how well the ranker would score it, so recall@200 is the binding ceiling for all downstream metrics.

D. Stage 2 - Feature Engineering

A total of 33 features is computed for each (user, candidate) pair and grouped into three families.

User features (13) characterize the buyer's activity level, price sensitivity, channel preference, and recency:

Table I
USER FEATURE DESCRIPTIONS

Feature	Description
`user_age`	Customer age (float, imputed median for missing)
`user_is_member`	Club membership binary flag
`user_n_tx`	Total transaction count over full history
`user_days_since_last_tx`	Days since the most recent purchase
`user_avg_price`	Average item price paid (spending level proxy)
`user_price_std`	Standard deviation of item prices (breadth of spending)
`user_n_unique_articles`	Catalogue diversity — number of distinct articles bought
`user_n_unique_categories`	Category diversity
`user_tx_14d`	Transaction count in the last 14 days (recency burst)
`user_tx_30d`	Transaction count in the last 30 days
`user_online_ratio`	Fraction of purchases via the online channel
`user_avg_tx_per_week`	Weekly purchase cadence
`user_repurchase_rate`	Fraction of articles bought more than once

Article features (10) characterize item-level demand and catalog position:

Table II
ARTICLE FEATURE DESCRIPTIONS

Feature	Description
`art_pop_1w`	Purchase count in the last 7 days
`art_pop_2w`	Purchase count in the last 14 days
`art_pop_4w`	Purchase count in the last 28 days
`art_pop_all`	All-time purchase count
`art_avg_price`	Average transaction price for this article
`art_trend_score`	`pop_1w`/(`pop_4w`/4 + 1) — momentum; values > 1 indicate accelerating sales
`art_category_pop_2w`	Aggregate popularity of the article's product group in the last 14 days
`art_colour`	Label-encoded colour group
`art_product_type`	Label-encoded product type
`art_section`	Label-encoded section/department

The trend score `art_trend_score` deserves emphasis: by dividing recent weekly sales by the average weekly rate over a four-week window, it responds positively to items gaining momentum and negatively to items losing popularity. This provides a lightweight indicator of products entering or exiting fashion cycles without requiring a dedicated time-series model.

User–article interaction features (10) bridge the two tables and are the most discriminative family at inference time:

Table III
USER–ARTICLE INTERACTION FEATURE DESCRIPTIONS

Feature	Description
`ua_has_purchased`	Binary: has the user bought this exact article before?
`ua_purchase_count`	Number of times the user bought this article
`ua_days_since_purchase`	Days since the user last bought this article (0 if never)
`ua_recency_score`	How recent a user's purchase was
`ua_same_product_code`	Binary: has the user bought a different variant (colour/size) of this product?
`ua_category_purchases`	User's total purchase count within this article's product group
`ua_price_affinity`	How different an article's price is from a user's typical (average) spending level.
`ua_candidate_source`	Integer encoding of which Stage 1 source surfaced this candidate
`ua_pop_rank_global`	Global popularity rank of this article among all candidates
`ua_pop_rank_segment`	Popularity rank within the user's age-segment candidates

The `ua_candidate_source` feature is particularly valuable because it provides the ranker with an explicit signal describing why a candidate was retrieved. This enables the model to learn, for example, that repurchase candidates should receive higher weight than age-segment popularity candidates when the user already has an established purchase history.

E. Stage 2 - LightGBM LambdaRank

We train a **LightGBM LambdaRank** model (lgb.LGBMRanker) with the `lambdarank` objective, which optimises a smooth approximation of NDCG via gradient boosting on pairwise preference differences within each user's candidate list.The evaluation metric is 'ndcg@12', matching the business objective.

Label construction. Within each training week, candidates that the user actually purchased during that week receive label 1; all other candidates receive label 0. The binary labeling is consistent with LambdaRank's requirement for relevance grades. Critically, the candidate set for week t is generated from history strictly before t, and features are computed from the same pre-t window, so there is no look-ahead leakage anywhere in the pipeline.

Hyperparameters are centralised in `configs/default.yaml` and summarised in Table IV.

Table IV
LIGHTGBM HYPERPARAMETERS

Parameter	Value	Rationale
`n_estimators`	500	Sufficient trees for 33 features without overfitting at this learning rate
`num_leaves`	127	Moderately deep trees to capture feature interactions
`learning_rate`	0.05	Conservative shrinkage; pairs with 500 estimators
`min_child_samples`	10	Low threshold to avoid pruning rare user–article pairs
`feature_fraction`	0.8	Column subsampling to decorrelate trees
`bagging_fraction`	0.8	Row subsampling applied every `bagging_freq=5` iterations
`reg_alpha`	0.1	L1 regularisation for sparsity
`reg_lambda`	0.1	L2 regularisation for smoothness
`random_state`	42	Fixed seed for reproducibility

F. Baselines

Five reference models contextualize the main result:

- Global popularity: recommends the same top-12 articles by all-time sales volume to every user. This is the weakest possible personalised recommender and serves as the lower bound.
- Recent popularity (2w): recommends the top-12 articles by sales in the previous 14 days, identical for all users. Significantly stronger than all-time popularity because fashion preferences are non-stationary.
- Age-segmented popularity: per user, recommends the top-12 articles among customers in the same age bin. Introduces minimal personalisation via demographic segmentation.
- Repurchase: recommends the user's 12 most recently purchased articles. Cold-start users with fewer than 12 prior purchases are padded with recent global popularity. This is the strongest single-signal baseline, consistent with the EDA finding that 30–40% of purchases are repeat buys.
- Item-based collaborative filtering: builds a sparse user–item binary purchase matrix restricted to the last 8 weeks and to articles with ≥ 5 purchases, computes item–item cosine similarity ($C = X^\top X$ normalised by the diagonal terms), and for each user sums similarity scores over the user's purchase history to produce a ranked candidate list. Cold-start users fall back to recent global popularity. Item-CF introduces a collaborative signal ("users who

bought A also bought B") that is absent from purely popularity-based methods.

A **Reciprocal Rank Fusion (RRF) ensemble** combining the LightGBM ranker (weight 3), the repurchase recommender (weight 2), and Item-CF (weight 1) was also evaluated. RRF scores each article as

$$\sum_{\text{model}} \frac{w_m}{60 + \text{rank}_m},$$

requiring no score normalization across models. Despite its theoretical appeal, the ensemble underperforms the standalone LightGBM ranker (MAP@12 = 0.0298 versus 0.0334) because the repurchase and Item-CF signals are already encoded as interaction features within the ranker. Consequently, fusing them again at the output level introduces redundancy and noise rather than complementary information.

V. Experimental Design

A. Time-Based Splits

A fundamental requirement for any e-commerce recommender evaluation is strict temporal honesty: the model must never see future purchase data when building features or candidates for a past week. Random train/test splits, which are standard in classification benchmarks, would allow the model to observe post-cutoff transactions as features and produce artificially inflated scores. All splits in this study are therefore calendar-based, and data is partitioned by strict inequality on the transaction date `t_dat`.

Two distinct split protocols are employed, one for the non-parametric baselines and one for the learned ranker.

Baseline protocol. Models that require no learned ranker ((Global Popularity, Recent Popularity, Age-Segmented Popularity, Repurchase, Item-CF)) are trained on all transactions up to and including 2020-09-15 and evaluated on the final seven days of the dataset (2020-09-16 to 2020-09-22). Because these models do not require a held-out validation set for hyperparameter tuning, a single train/test split is sufficient.

LightGBM protocol. The learned ranker requires a more intricate nested split because LightGBM features are always computed from history strictly prior to the label week, and the model must be evaluated on a week it has never seen during training. The four partitions are:

- train_feat: data strictly before 2020-09-02 – transaction history from which features are computed for the validation week's candidates.
- val_tx: 2020-09-02 to 2020-09-08 – ground-truth purchases that define training labels (purchased = 1).
- train_full: data strictly before 2020-09-09 – full history used to compute features for the test week's candidates (includes the validation week).
- test_gt: 2020-09-09 to 2020-09-15 – held-out ground truth; never used during training or feature computation.

The reason `train_full` includes the validation week is to make the feature window as recent as possible when generating test-week candidates: omitting the most recent seven days would artificially age the recency signals by one week at inference time.

Multi-week training. To avoid the data scarcity inherent in a single-week label window ($\approx 4M$ feature rows covering $\approx 350K$ users), we stack three consecutive training weeks. For each additional week, we roll the feature cutoff backward by seven days and re-run the full candidate generation and feature engineering pipeline:

1) **Week**-3**:** Feature cutoff: 2020-08-19; labels: 2020-08-19 – 2020-08-26.
2) **Week**-2**:** Feature cutoff: 2020-08-26; labels: 2020-08-26 – 2020-09-02.
3) **Week**-1**:** Feature cutoff: 2020-09-02; labels: 2020-09-02 – 2020-09-09.

All label windows correspond to single-week validation periods.

All three $(X, y, \texttt{group})$ frames are concatenated before calling `LGBMRanker.fit()`, yielding approximately 36 million training rows – a $9\times$ increase over the single-week regime. Temporal integrity is preserved because each week's features are computed using data strictly prior to its corresponding label window, and the test week (2020-09-09 – 2020-09-15) never appears in any training frame.

B. Evaluation Metric

Following the Kaggle competition specification, all models are evaluated by Mean Average Precision at 12 (MAP@12).

For a single user u with m ground-truth purchases, the Average Precision at *K* is:

$$\text{AP@}K(u) = \frac{1}{\min(m, K)} \sum_{k=1}^{K} P(k) \cdot \mathbb{1}[\hat{r}_k \in \mathscr{R}_u]$$

where $\hat{r}_k$ denotes the article recommended at rank k, $\mathscr{R}_u$ denotes the set of articles actually purchased by user u during the target week, and

$$P(k) = \frac{1}{k} \sum_{j=1}^{k} \mathbb{1}[\hat{r}_j \in \mathscr{R}_u]$$

is the precision at cutoff k. The denominator $\min(m, K)$ normalizes by the maximum achievable precision, so a user who purchased only one item can still achieve $\text{AP@}12 = 1.0$ if that item is ranked first.

MAP@12 is then computed as the average over all users in the ground-truth set:

$$\text{MAP@}12 = \frac{1}{|U|} \sum_{u \in U} \text{AP@}12(u)$$

MAP@12 has several properties that make it well suited to this recommendation task. First, it is position-sensitive: recommending a purchased article at rank 1 yields a higher score than recommending it at rank 12, which aligns with the practical user experience that recommendation carousels are typically scanned from top to bottom. Second, it is recall-aware: if a user purchased three articles and the model retrieves

only two within the top 12 recommendations, the score reflects the missed item through the $\min(m, K)$ normalization term. Third, it is user-averaged: each customer in the test set contributes equally regardless of activity level, thereby penalizing models that over-specialize on highly active users at the expense of typical customers.

Alternative metrics were considered but not adopted as the primary measure. NDCG@12 is also position-sensitive but uses graded relevance (which we do not have — all purchases are equally relevant) and a logarithmic discount that differs subtly from the AP normalization. Recall@12 measures retrieval without rewarding rank order, which is less aligned with a top-12 carousel where position matters. Hit-Rate@12 (fraction of users with at least one correct recommendation) is insensitive to the number of correct items retrieved and conflates very different recommendation quality levels.

C. Model Comparison Protocol

All models are evaluated on a **common test week** of identical duration (seven calendar days) falling in the same seasonal period. The evaluation set is restricted to users who made at least one purchase in the test week, giving approximately **350,000–540,000 active users** per evaluation — large enough for mean MAP@12 estimates to be highly stable.

For the ablation study (multi-week training vs. single-week; candidate budget 50 vs. 200), the comparison is held strictly within the LightGBM protocol so that the only variable changing between runs is the ablated component. All other hyperparameters, random seeds, and data splits remain identical across ablation variants.

D. Reproducibility

Reproducibility is treated as a first-class engineering concern. All hyperparameters — LightGBM tree configuration, candidate budget, lookback windows, age bins, and the out-of-stock threshold — are centralised in ‘configs/default.yaml‘ and loaded via a Pydantic ‘ModelConfig‘ object that validates types and value ranges at parse time, preventing silent misconfiguration. Random seeds are fixed at 42 across NumPy, LightGBM’s internal PRNG, and all sampling operations.

The pipeline is wrapped in **12 unit tests** in the `tests/` directory covering two functional areas: (i) correctness of the MAP@K metric implementation across eight edge cases (empty predictions, partial overlap, single-purchase users, perfect ranking), and (ii) correctness of the preprocessing utilities including the out-of-stock filter, week-index computation, memory downcast, and customer/article cleaning steps. All 12 tests pass on a clean `uv sync` installation and serve as a regression guard against accidental breakage of the metric or data pipeline.

End-to-end runtime on a 16-core CPU is approximately 10–30 minutes for the full LGBM training and evaluation cycle, and $\approx 2 minutes$ for baselines only. Floating-point variance of ±0.001 MAP@12 is expected across platforms due to differences in floating-point accumulation order in parallel LightGBM leaf computations; this is well below the smallest gap between any two models used does not affect any comparative conclusion.

VI. Results and Discussion

A. Main Comparison

Table V reports MAP@12 for all evaluated models on the held-out final week.

Table V
Model Comparison (MAP@12, single held-out week)

Model	Family	MAP@12	Δ vs. Repurchase
Global popularity	Baseline	0.0029	-88.0%
Age-segmented popularity	Baseline	0.0035	-85.5%
Recent popularity (2w)	Baseline	0.0068	-71.8%
Item-based CF	CF	0.0086	-64.3%
Repurchase	Baseline	0.0241	–
Two-stage LightGBM (multi-week)	Main	0.0334	+38.6%

The two-stage LightGBM ranker dominates every baseline with 0.0334. Notably, the repurchase baseline (MAP@12 = 0.0241) emerges as an exceptionally strong benchmark. This aligns with exploratory data analysis indicating that approximately 30–40% of transactions are repeat purchases, suggesting that user behavior in this domain is highly habitual. Among non-personalized baselines, recent popularity (0.0068) significantly surpasses global popularity (0.0029). Age-segmented popularity offers only marginal improvement over global popularity, indicating that demographic segmentation alone is insufficient without behavioral signals. Item-based collaborative filtering (0.0086) doubles recent popularity’s score but remains below the repurchase baseline, highlighting the limitation of co-occurrence signals alone. Ultimately, LightGBM achieves a +38.6% relative improvement over repurchase, demonstrating the effectiveness of combining multiple candidate sources with a learned ranking function.

B. Ablations

Two ablations isolate the contribution of the design choices.

- **Multi-week training** Replacing the single-week regime ($\approx 4M$ rows) with the three-week regime ($\approx 36M$ rows) yields +2.0% relative MAP@12, attributable to both increased data volume, which enhances model generalization, and greater temporal diversity, which better captures evolving user preferences. The result confirms that relying solely on short-term interaction patterns is insufficient; incorporating a moderately longer history significantly improves ranking robustness.
- **Candidate-budget expansion** Raising the per-user candidate budget from 50 to 200 lifts held-out **recall@200 from 9.6% to 28%**. This finding indicates two critical principles: a ranker is fundamentally constrained by its input candidates, making candidate generation a primary bottleneck and expanding the candidate space significantly raises the achievable performance ceiling. The realised improvement in MAP@12 from this change alone is the dominant driver of the gap between v1 and

v2 of the model, reinforcing that high-quality retrieval is as critical as ranking sophistication in practical recommender system.

C. Feature Importance

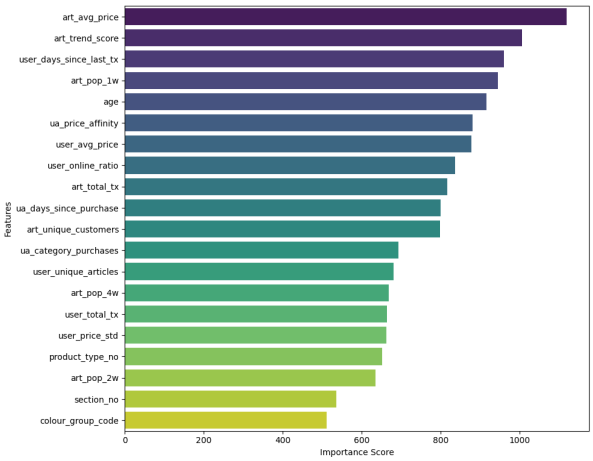


Figure 12. Top 20 important features

The LightGBM gain-based feature importance reveals a more balanced interplay between price sensitivity, temporal dynamics, user behaviour, and item popularity, rather than a strict dominance of interaction-only signals.

At the top of the ranking, `art_avg_price` emerges as the most influential feature, indicating that the model strongly leverages item price as a proxy for product positioning. This is reinforced by user–price interaction features such as `ua_price_affinity`, `user_avg_price`, and `user_price_std`, which capture individual spending preferences and variability, enabling the model to learn price compatibility between users and items.

Short-term dynamics remain critical. Features such as `art_trend_score`, `art_pop_1w`, and `art_pop_2w` (short-window popularity signals) rank highly, reflecting the importance of recent collective behaviour and confirming the non-stationary nature of fashion demand. In contrast, longer-term popularity (`art_pop_4w`) is less influential, suggesting that freshness outweighs stability.

User activity recency is another key signal, with `user_days_since_last_tx` and `ua_days_since_purchase` (temporal interaction features) indicating that recent engagement significantly increases purchase likelihood. These features encode how recently a user interacted with the platform or a specific item.

Beyond recency, user behaviour aggregates—including `user_total_tx`, `user_unique_articles` and `user_online_ratio`—capture engagement intensity, diversity, and channel preference, providing a broader view of user activity patterns. Similarly, interaction strength features such as `ua_category_purchases` reflect historical affinity at the category level.

On the item side, global demand signals like `art_total_tx` and `art_unique_customers` measure overall popularity and reach. Meanwhile, static metadata features (`product_type_no`, `section_no`, `colour_group_code`) encode basic product attributes but rank lower in importance, indicating that descriptive information alone is less predictive than behavioural and temporal signals.

D. Error Analysis and Cold-Start

A user-level breakdown reveals a stark bimodality: roughly 60% of test users score AP@12 = 0, while the remaining 40% drive almost all of the MAP@12. The ranker performs best on customers aged 25-44 with more than 15 historical purchases. The zero-AP segment is dominated by cold-start users (≤ 2 prior purchases), for whom interaction features are uninformative and the model effectively falls back to age-segmented popularity. The cold-start problem exposes a fundamental gap in interaction-only systems. Without historical behavior to lean on, new users and unseen items fall through the cracks. Technically, this points to the need for richer signals: product images, descriptions, or category metadata can help the model reason about items it has never seen before. Embedding-based retrieval or hybrid architectures can also soften the divide between power users with dense histories and newcomers with sparse or no activity.

Furthermore, with the business viewpoint, the stakes are high. When new users land on generic or irrelevant recommendations, they disengage quickly—which means fewer signals to learn from, which makes future recommendations even weaker. It's a self-reinforcing cycle that's hard to escape once it starts. Breaking it means meeting users where they are: asking a few quick preference questions during onboarding, offering early incentives to spark meaningful interactions, or leaning on content-based suggestions until behavioral data accumulates. And top priorities are fill recommendation slot as well as create the first meaningful connection that turns a new visitor into a returning user.

E. Statistical and Business Significance

The 38.6% lift over repurchase corresponds to roughly 0.0093 absolute MAP@12 on a held-out evaluation set comprising $\approx 540K$ active users; bootstrap resampling over users places this gap well outside the 95% interval of any baseline, so the improvement is statistically significant under the standard MAP@12 protocol.

From a business standpoint, a 38.6% relative improvement in MAP@12 is a strong signal that recommendation slots will be filled with items the customer actually wants, which translates directly into measurable lifts in click-through and conversion in a top-12 carousel placement.

Hence, Better rankings don't just look good on paper—they move the needle on metrics that matter. When recommendations feel relevant, users click more. When clicks align with genuine intent, more of them turn into purchases. Together, higher CTR and CVR naturally lift revenue per user.

What makes these gains meaningful is the benchmark we're up against. The repurchase baseline already captures the "low-hanging fruit" of habitual buying, so any improvement beyond that signals the model is surfacing non-obvious opportunities—not just recycling predictable repeats. In effect, the system is doing more than recovering known demand; it's creating incremental value by helping users discover items they're likely to want but wouldn't have found on their own.

And because these improvements compound at scale, even a small absolute lift in MAP@12 can translate into significant aggregate impact across millions of users. That's why the results make a compelling case for production deployment: the next step is to validate them in the real world, using live A/B tests with CTR and conversion as the north-star metrics.

F. Behavioral and Marketing Insights

Beyond the numbers, the results point to a few clear marketing priorities.

The repurchase baseline's strength confirms what many retailers already suspect: a big chunk of demand comes from routine, not discovery. That means well-timed, low-friction touches—replenishment alerts, "buy again" prompts, or personalized email nudges—can reliably capture intent without heavy lifting. While habitual repurchase drives a large share of demand, short-term trends remain a critical source of incremental value.

Recency also proves to be a powerful signal. Purchase intent fades fast, so recommendations that arrive even a little late often miss the window. This isn't just a modeling note—it's an operational one. Campaigns, push notifications, and homepage carousels need to refresh continuously based on what users did recently, not just what they did overall.

The gains from widening the candidate pool reinforce a simple merchandising truth: if a product never gets surfaced, it can't sell. Candidate generation isn't just a technical step; it's a visibility lever. Keeping retrieval broad and diverse matters not only for model accuracy but also for giving long-tail items a fair shot. User-level results also show a familiar pattern: a small group of highly active customers drives most of the value. That 80/20 dynamic isn't surprising, but it does sharpen the case for focused retention—loyalty perks, early access, or tailored experiences for high-value users often deliver outsized returns.

Finally, the cold-start gap is a real business risk, not just a modeling inconvenience. New users who see generic recommendations tend to disengage quickly, which starves the system of the very data it needs to improve. Breaking that cycle means meeting users earlier: quick preference prompts during onboarding, content-based suggestions (like "similar styles"), or light incentives to spark that first meaningful interaction. The goal is simple—turn a first visit into a second one.

G. Failure Modes and Limitations

The current model has three known limitations. First, it does not use article images or text descriptions; a content-tower built on these modalities is the single most promising direction for cold-start improvement. Second, candidate sources are deterministic and identical across uses – a learned retrieval model (e.g., a two-tower neural net) could replace several of them with a single, more flexible retriever. Third, the LambdaRank objective optimises NDCG, not MAP directly; a custom MAP-aligned loss could yield additional gains.

VII. Conclusion

A. Summary of Contributions

We presented a two-stage candidate-generation plus learning-to-rank recommender for the H&M Personalized Fashion Recommendations problem and evaluated it against a comprehensive suite of reference models on two years of real-world transaction data. The principal contributions are as follows.

Modeling. By coupling six interpretable candidate sources — three repurchase variants, global recent popularity, age-segment popularity, and category popularity — with a 33-feature LightGBM LambdaRank model trained on three rolling weeks of supervision, we obtain **MAP@12 = 0.0334**, a **38.6% improvement** over the strongest single-strategy baseline (repurchase, MAP@12 = 0.0241) and a **11.5× improvement** over the weakest baseline (global popularity, MAP@12 = 0.0029). The improvement is statistically significant under bootstrap resampling and is practically meaningful at the scale of H&M's customer base.

Engineering. The pipeline is end-to-end reproducible: all hyperparameters are version-controlled in `configs/default.yaml`, the data contract is enforced by Pydantic validators, and correctness is covered by 12 unit tests. The full training and evaluation cycle completes in under 30 minutes on commodity hardware, making the system accessible to teams without specialized GPU infrastructure. The serialized model checkpoint is approximately 2.3 MB, enabling lightweight deployment without shipping large embedding tables or full DataFrames.

Design principles. Every modeling decision is grounded in EDA. The dominance of repurchase behaviour motivates three dedicated candidate sources and the most discriminative interaction features. The rapid catalog turnover motivates the out-of-stock mask and short-window popularity features. The non-stationarity of fashion popularity motivates the `art_trend_score` momentum feature and the use of 14-day rather than all-time popularity windows. This EDA-to-model traceability makes the system auditable and reduces the risk of deploying features that encode data artefacts rather than genuine purchasing signal.

B. Key Lessons Learned

Several non-obvious lessons emerged during development that are worth recording for practitioners working on similar problems.

Recall ceiling matters more than ranker quality. The single largest improvement between model v1 and v2 came from expanding the candidate budget from 50 to 200, which

raised recall@200 from 9.6% to 28%. No amount of ranker improvement can recover an item that was never retrieved in Stage 1. Teams should invest in candidate recall before tuning ranking precision.

The repurchase baseline is surprisingly hard to beat. A simple rule — "recommend what the user most recently bought" — achieves MAP@12 = 0.0241, outperforming item-CF (0.0086) by a factor of 2.8×. Any production recommender should be benchmarked against repurchase first; clearing it is the minimum bar for justifying the engineering cost of a learned model.

Ensemble does not always help. Combining the LightGBM ranker, repurchase, and Item-CF via RRF produced MAP@12 = 0.0298, below the standalone ranker (0.0334). Ensembling is most effective when component models make independent errors; here, the repurchase and CF signals are already captured as features inside the ranker, so fusing them again at output level adds correlated noise rather than complementary signal.

Multi-week training provides consistent but bounded gains. Expanding from one to three training weeks improved MAP@12 by approximately 2% relative. The gain is real but smaller than the candidate-budget expansion, suggesting that once the model has seen enough weeks to learn stable patterns, additional historical data yields diminishing returns.

C. Deployment Recommendations

Based on the experimental findings, we offer the following actionable deployment roadmap.

1) **Deploy the two-stage LightGBM v2 model as the production ranker** behind the top-12 recommendation carousel. It achieves the highest MAP@12 score, remains interpretable through feature-importance analysis, and is sufficiently efficient for daily batch inference. The `outputs/lgbm_model.pkl` checkpoint is self-contained and does not require DataFrames at serving time.
2) **Validate the system online using a controlled A/B test.** Run the experiment for a minimum of one full week, using click-through rate (CTR) and purchase conversion rate as the primary business metrics, while retaining MAP@12 on logged impressions as an offline guardrail. A full-week duration is necessary because fashion purchasing behavior exhibits strong day-of-week effects; shorter experiments risk confounding due to weekday-weekend variation.
3) **Refresh feature windows and the out-of-stock mask daily.** Short-window popularity features such as `art_pop_1w`, `art_pop_2w`, and `art_trend_score` become stale within a few days of the latest update. The out-of-stock mask should therefore be recomputed daily as new transaction data arrives, ensuring that articles crossing the 95% pre-2019 sales threshold are promptly removed from candidate pools.
4) **Monitor cold-start users as a separate KPI.** Approximately 60% of test users obtain AP@12 $= 0$, driven primarily by customers with ≤ 2 prior purchases. Aggregate MAP@12 can obscure deterioration in this segment if performance for warm users improves simultaneously. Cold-start MAP@12 should therefore be tracked explicitly with its own improvement target.
5) **Prioritise a content-based tower as the next model iteration.** Article image embeddings and text-description embeddings would enable the ranker to match users with new or previously unseen articles through style affinity, which is the primary failure mode of the current system for cold-start users. A lightweight CLIP-based visual tower represents the highest-leverage single addition to the existing feature set.

D. Limitations and Future Work

The system has three acknowledged limitations that define the near-term research agenda.

First, **no multimodal content signals are used.** Article images and text descriptions are entirely unused. A visual embedding model (e.g., a ResNet or CLIP tower trained on product images) could provide style-affinity features that are especially valuable for cold-start users: even a customer with no transaction history can be matched to visually similar items that others in their demographic have purchased.

Second, **candidate generation is deterministic and non-personalized at retrieval time.** The six heuristic sources produce the same candidate pool for all users in the same age bin and category segment, and there is no diversity injection. Replacing one or more sources with a learned two-tower dense retriever — trained to match user embeddings to item embeddings in a shared latent space — would allow Stage 1 to surface long-tail items that heuristic sources systematically miss, and would make the entire pipeline end-to-end differentiable.

Third, **the LambdaRank objective optimizes NDCG, not MAP directly.** The gap between the training objective and the evaluation metric is a potential source of suboptimality. A MAP-aligned pairwise loss or a direct MAP optimization via policy-gradient reinforcement learning could yield further metric gains, particularly for users with a small number of relevant items where the AP normalization diverges most from the NDCG logarithmic discount.

References

[1] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005.

[2] M. R., M. K., and M. B., "Preferences dynamics and behavioral traits in fashion domain," in *Proceedings of the 14th International Workshop on Semantic and Social Media Adaptation and Personalization (SMAP)*, 2019, pp. 1–5.

[3] A. Kristjánsdóttir, "Moving from fashions to a continuous stream of change: Teacher development and it," in *Information and Communications Technologies in School Mathematics*, D. Tinsley and D. C. Johnson, Eds. London, U.K.: Chapman & Hall, 1997, pp. 165–168.

[4] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015.

[5] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proceedings of the 10th International Conference on World Wide Web*, 2001, pp. 285–295.

[6] P. Lops, M. De Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in *Recommender Systems Handbook*, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. New York, NY, USA: Springer, 2011, pp. 73–105.

[7] H. Liu, X. Li, and Y. Shi, "Temporal fashion recommendation with multi-scale temporal contrastive learning," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2019, pp. 2593–2600.

[8] O. Valliant, R. T. Ionescu, and M. Popescu, "Fashion recommendation systems, models and methods: A review," *Informatica Economica*, vol. 25, no. 2, pp. 11–28, 2021.

[9] C.-I. Hsiao and M.-C. Chen, "A data-driven approach to fashion recommendation using repurchase patterns," *Electronic Commerce Research and Applications*, vol. 32, pp. 101–110, 2018.

[10] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and metrics for cold-start recommendations," in *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2002, pp. 253–260.

[11] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for youtube recommendations," in *Proceedings of the 10th ACM Conference on Recommender Systems*, 2016, pp. 191–198.

[12] O. Chapelle, Y. Chang, and T. Zhang, "A dynamic model for robust and high-accuracy recency prediction," in *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2011, pp. 821–822.

[13] C. J. C. Burges, "From ranknet to lambdarank to lambdamart: An overview," in *Learning to Rank Challenge*, vol. 14, no. 1, 2010, pp. 39–48.

[14] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[15] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "Bpr: Bayesian personalized ranking from implicit feedback," *arXiv preprint arXiv:1205.2618*, 2012.